GENERALITAT VALENCIANA
Conselleria d'Educació, Cultura i Esport
IES JUAN DE GARAY
C/ Juan de Garay, 25 46017 Valencia
Tel: 961 20 60 45 Fax: 961 20 60 46
46012963@gva.es www.juandegaray.es
GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL
Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

UD 07.2.
JSON 2

LM
Ciclos Formativos Grado Superior

Pascual Ligero
pa.ligero2@edu.gva.es
2025/2026
Versión:260421.1100

Licencia



Reconocimiento – NoComercial – CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

> ⚠ Importante

> ⚛ Interesante

ÍNDICE DE CONTENIDO

1. JSON AGAIN.....5
1.1 Introducción.....5
1.2 Qué es JSON.....5
1.3 JSON en JavaScript.....5
1.4 Ver todas las claves.....6
1.5 Ver todos los valores.....6
1.6 Ver claves y valores.....7
1.7 Si empieza por llaves.....7
1.8 Si empieza por corchetes.....8
1.9 Uso de for.....8
1.10 Uso de foreach,.....9
1.11 Uso de for...in.....9
1.12 Uso de for...of.....10
1.13 Diferencias rápidas.....11
2. Ejemplos prácticos completos.....11
2.1 Ejemplo 1: JSON objeto.....11
2.2 Ejemplo 2: JSON array de objetos.....11
2.3 Ejemplo 3: Ver claves y valores de cada objeto dentro del array.....12
2.4 Errores comunes.....12
3. Ejercicios.....12
3.1 Ejercicio 1.....12
3.2 Ejercicio 2.....13
3.3 Ejercicio 3.....13
3.4 Ejercicio 4.....13
4. Soluciones.....13
4.1 Solución 1.....13
4.2 Solución 2.....14
4.3 Solución 3.....14
4.4 Solución 4.....14
4.5 Regla fácil de recordar.....15

UD07.2. JSON 2

1. JSON AGAIN

1.1 Introducción

En JavaScript, un JSON puede representar **un objeto** si empieza por `{}` o **un array** si empieza por `[]`, y la forma de recorrerlo cambia según eso. Para ver **claves** y **valores**, lo más importante es distinguir si estás trabajando con un objeto o con una lista de objetos.

1.2 Qué es JSON

JSON es un formato de datos basado en pares **clave: valor**, y en JavaScript normalmente lo conviertes desde texto a un valor utilizable con `JSON.parse()`.

Puede empezar de tres formas:

- `{ ... }` → representa un **objeto**.
- `[ ... ]` → representa un **array** o lista.
- Viene de un fetch: → convierto en un JSON

1) Ejemplo de JSON que empieza por `{}`:

```
json
{
  "nombre": "Lucía",
  "edad": 22,
  "activo": true
}
```

2) Ejemplo de JSON que empieza por `[]`:

```
json
[
  { "nombre": "Ana", "edad": 20 },
  { "nombre": "Luis", "edad": 25 }
]
```

3) Ejemplo de JSON `desde una promesa Fetch`:

```
javascript
fetch(url)
  .then(response => response.json())
  .then(datos => console.log(datos));
```

1.3 JSON en JavaScript

Si el JSON **viene como texto, primero hay que convertirlo con `JSON.parse()`**, porque JSON como tal es una cadena y luego pasa a ser un objeto o array nativo de JavaScript.

```
javascript
const textoJSON = '{"nombre":"Lucía","edad":22,"activo":true}';
const datos = JSON.parse(textoJSON);

console.log(datos.nombre); // Lucía
console.log(datos.edad);   // 22
```

Si quieres volver a **convertir un objeto JavaScript a texto JSON, usas `JSON.stringify()`**.

```
javascript
const usuario = {
  nombre: "Carlos",
  edad: 30
};

const jsonTexto = JSON.stringify(usuario);
console.log(jsonTexto);
```

1.4 Ver todas las claves

Para sacar todas las claves de un objeto, una forma muy práctica es **`Object.keys`**`(objeto)`, que devuelve un array con los nombres de las propiedades.

```
javascript
const persona = {
  nombre: "Marta",
  edad: 28,
  ciudad: "Valencia"
};

const claves = Object.keys(persona);
console.log(claves); // array de claves ["nombre", "edad", "ciudad"]
```

Si quieres mostrarlas una por una, puedes recorrer ese array.

```
javascript
const persona = {
  nombre: "Marta",
  edad: 28,
  ciudad: "Valencia"
};

Object.keys(persona).forEach(function(clave) {
  console.log(clave);
});
```

1.5 Ver todos los valores

Para ver solo los valores, puedes usar `Object.values(objeto)`, que devuelve un array con los valores del objeto. La idea de extraer claves para trabajar con la estructura del objeto aparece en ejemplos prácticos de manipulación JSON con objetos JavaScript.

```
javascript
const persona = {
```

```
  nombre: "Marta",
  edad: 28,
  ciudad: "Valencia"
};

const valores = Object.values(persona);
console.log(valores); // ["Marta", 28, "Valencia"]
```

Si quieres mostrar valor por valor:

```
javascript
Object.values(persona).forEach(function(valor) {
  console.log(valor);
});
```

1.6 Ver claves y valores

Cuando quieres mostrar **clave y valor juntos**, una opción muy usada es `for...in`, que recorre las propiedades de un objeto.

```
javascript
const persona = {
  nombre: "Marta",
  edad: 28,
  ciudad: "Valencia"
};

for (let clave in persona) {
  console.log(clave + ": " + persona[clave]);
}
```

Salida:

```
javascript
nombre: Marta
edad: 28
ciudad: Valencia
```

También puedes hacerlo con `Object.keys()`:

```
javascript
Object.keys(persona).forEach(function(clave) {
  console.log(clave + ": " + persona[clave]);
});
```

1.7 Si empieza por llaves

Si el JSON empieza por `{`, normalmente tendrás un **objeto**, así que puedes acceder directamente a sus propiedades por nombre.

```
javascript
const textoJSON = '{"nombre":"Pedro","edad":35,"trabajo":"Técnico"}';
const datos = JSON.parse(textoJSON);

console.log(datos.nombre);
console.log(datos.edad);
```

```
console.log(datos.trabajo);
```

Para ver todas sus claves y valores:

```
javascript
for (let clave in datos) {
  console.log("Clave:", clave);
  console.log("Valor:", datos[clave]);
}
```

1.8 Si empieza por corchetes

Si el JSON empieza por `[`, normalmente tendrás un **array**, y dentro puede haber números, textos u objetos.

```
javascript
const textoJSON = `
[
  { "nombre": "Ana", "edad": 20 },
  { "nombre": "Luis", "edad": 25 },
  { "nombre": "Sofía", "edad": 23 }
]
`;

const personas = JSON.parse(textoJSON);
console.log(personas);
```

Para recorrer ese array completo, puedes usar `for`, `forEach()` o `for...of`, que son formas habituales de iterar arrays en JavaScript.

1.9 Uso de for

El `for` clásico es muy útil cuando quieres controlar el índice, empezar en una posición concreta o avanzar como quieras.

`Usar for`

- Cuando necesitas el índice.
- Cuando quieres parar con `break`.
- Cuando quieres recorrer solo una parte del array.

```
javascript
const nombres = ["Ana", "Luis", "Sofía"];

for (let i = 0; i < nombres.length; i++) {
  console.log("Posición:", i);
  console.log("Valor:", nombres[i]);
}
```

Con un JSON que empieza por `[]`:stackoverflow+1

```
javascript
const personas = [
  { nombre: "Ana", edad: 20 },
```

```
  { nombre: "Luis", edad: 25 },
  { nombre: "Sofía", edad: 23 }
];

for (let i = 0; i < personas.length; i++) {
  console.log(personas[i].nombre);
  console.log(personas[i].edad);
}
```

1.10 Uso de foreach,

`forEach()` es un método de los arrays y sirve para ejecutar una función por cada elemento. Se usa mucho cuando solo quieres recorrer y mostrar datos de forma clara.

Cuándo usar `forEach`

- Cuando ya sabes que estás recorriendo un array.
- Cuando no necesitas usar `break` ni `continue` de forma tradicional.
- Cuando buscas un código legible y directo.
-

```
javascript
const nombres = ["Ana", "Luis", "Sofía"];

nombres.forEach(function(nombre, indice) {
  console.log(indice + ": " + nombre);
});
```

Con un array de objetos JSON:

```
javascript
const personas = [
  { nombre: "Ana", edad: 20 },
  { nombre: "Luis", edad: 25 },
  { nombre: "Sofía", edad: 23 }
];

personas.forEach(function(persona) {
  console.log(persona.nombre + " tiene " + persona.edad + " años");
});
```

1.11 Uso de for...in

`for...in` recorre las **claves** o propiedades de un objeto, por eso es muy útil para mostrar todos los campos de un JSON que empieza por `{}`.

Cuándo usar `for...in`

- Para objetos.
- Para listar claves de un JSON objeto.
- No suele ser la primera opción para arrays.

```
javascript
```

```
const coche = {
  marca: "Seat",
  modelo: "Ibiza",
  año: 2022
};

for (let clave in coche) {
  console.log(clave);         // marca, modelo, año
  console.log(coche[clave]);  // Seat, Ibiza, 2022
}
```

También puede usarse sobre arrays, pero normalmente no es la mejor opción si lo que quieres son los valores del array, porque `for...in` está orientado a propiedades o claves.

```
javascript
const colores = ["rojo", "verde", "azul"];

for (let indice in colores) {
  console.log(indice);          // 0, 1, 2
  console.log(colores[indice]); // rojo, verde, azul
}
```

1.12 Uso de for...of

`for...of` está pensado para recorrer directamente los **valores** de estructuras iterables, como arrays o strings, y suele ser más cómodo que `for` cuando no necesitas el índice.

Cuándo usar `for...of`

- Para arrays.
- Para strings.
- Cuando quieres cada valor directamente, sin manejar índices.

```
javascript
const colores = ["rojo", "verde", "azul"];

for (let color of colores) {
  console.log(color);
}
```

Con un JSON que empieza por `[]`:

```
javascript
const personas = [
  { nombre: "Ana", edad: 20 },
  { nombre: "Luis", edad: 25 }
];

for (let persona of personas) {
  console.log(persona.nombre);
  console.log(persona.edad);
}
```

1.13 Diferencias rápidas

Estructura	Qué recorre	Uso típico
`for`	Índices o contador	Arrays cuando necesitas control total
`forEach()`	Valores de un array	Recorrido simple y claro de arrays
`for...in`	Claves o propiedades	Objetos JSON que empiezan por `{}`
`for...of`	Valores	Arrays y colecciones iterables

2. EJEMPLOS PRÁCTICOS COMPLETOS

2.1 Ejemplo 1: JSON objeto

```
javascript
const textoJSON = `
{
  "nombre": "Elena",
  "edad": 26,
  "ciudad": "Valencia",
  "activo": true
}
`;

const usuario = JSON.parse(textoJSON);

// Ver todas las claves
console.log(Object.keys(usuario));

// Ver todos los valores
console.log(Object.values(usuario));

// Mostrar clave y valor
for (let clave in usuario) {
  console.log(clave + ": " + usuario[clave]);
}
```

2.2 Ejemplo 2: JSON array de objetos

```
javascript
const textoJSON = `
[
  { "nombre": "Ana", "nota": 8 },
  { "nombre": "Luis", "nota": 6 },
  { "nombre": "Marta", "nota": 9 }
]
`;

const alumnos = JSON.parse(textoJSON);

// Recorrer con for
for (let i = 0; i < alumnos.length; i++) {
  console.log(alumnos[i].nombre + " - " + alumnos[i].nota);
}
```

```
// Recorrer con forEach
alumnos.forEach(function(alumno) {
  console.log(alumno.nombre + " - " + alumno.nota);
});

// Recorrer con for...of
for (let alumno of alumnos) {
  console.log(alumno.nombre + " - " + alumno.nota);
}
```

2.3 Ejemplo 3: Ver claves y valores de cada objeto dentro del array

```
javascript
const alumnos = [
  { nombre: "Ana", nota: 8 },
  { nombre: "Luis", nota: 6 }
];

alumnos.forEach(function(alumno) {
  for (let clave in alumno) {
    console.log(clave + ": " + alumno[clave]);
  }
});
```

Este patrón de combinar un recorrido del array con otro de las propiedades del objeto es una forma habitual de recorrer estructuras JSON anidadas.

2.4 Errores comunes

- Confundir JSON con objeto JavaScript: JSON suele llegar como texto y necesitas `JSON.parse()` para usarlo como dato en JavaScript.
- Usar `for...in` para todo: funciona muy bien con objetos, pero para arrays suele ser mejor `for`, `forEach()` o `for...of`.
- Intentar acceder a una propiedad sin revisar la estructura: si empieza por `[]`, primero tienes que entrar al elemento del array.

Ejemplo:

```
javascript
const datos = [
  { nombre: "Ana" },
  { nombre: "Luis" }
];

console.log(datos[0].nombre); // correcto
```

3. EJERCICIOS

3.1 Ejercicio 1

Dado este objeto:

```
javascript
const producto = {
  nombre: "Teclado",
  precio: 25,
  stock: 10
};
```

Haz lo siguiente:

- Muestra todas las claves.
- Muestra todos los valores.
- Muestra `clave: valor` con `for...in`.

3.2 Ejercicio 2

Dado este array:

```
javascript
const numeros = [5, 10, 15, 20];
```

Haz lo siguiente:

- Recorre con `for`.
- Recorre con `forEach()`.
- Recorre con `for...of`.

3.3 Ejercicio 3

Dado este JSON:

```
javascript
const textoJSON = `
[
  { "nombre": "Pablo", "edad": 19 },
  { "nombre": "Sara", "edad": 21 }
]
`;
```

Haz lo siguiente:

- Convierte el texto a objeto JavaScript.
- Muestra solo los nombres.
- Muestra todas las claves y valores de cada persona.

3.4 Ejercicio 4

Dado este JSON objeto:

```
javascript
const textoJSON = '{"marca":"HP","ram":"16GB","ssd":"512GB"}';
```

Haz lo siguiente:

- Usa `JSON.parse()`.
- Muestra todas las claves con `Object.keys()`.
- Muestra todos los valores con `Object.values()`.

4. SOLUCIONES

4.1 Solución 1

```
javascript
```

```
const producto = {
  nombre: "Teclado",
  precio: 25,
  stock: 10
};

console.log(Object.keys(producto));
console.log(Object.values(producto));

for (let clave in producto) {
  console.log(clave + ": " + producto[clave]);
}
```

4.2 Solución 2

```
javascript
const numeros = [5, 10, 15, 20];

for (let i = 0; i < numeros.length; i++) {
  console.log(numeros[i]);
}

numeros.forEach(function(numero) {
  console.log(numero);
});

for (let numero of numeros) {
  console.log(numero);
}
```

4.3 Solución 3

```
javascript
const textoJSON = `
[
  { "nombre": "Pablo", "edad": 19 },
  { "nombre": "Sara", "edad": 21 }
]
`;

const personas = JSON.parse(textoJSON);

for (let persona of personas) {
  console.log(persona.nombre);
}

personas.forEach(function(persona) {
  for (let clave in persona) {
    console.log(clave + ": " + persona[clave]);
  }
});
```

4.4 Solución 4

```
javascript
const textoJSON = '{"marca":"HP","ram":"16GB","ssd":"512GB"}';

const ordenador = JSON.parse(textoJSON);

console.log(Object.keys(ordenador));
console.log(Object.values(ordenador));
```

4.5 Regla fácil de recordar

- Si ves `{}` piensa en **objeto** → usa mucho `for...in`, `Object.keys()` y `Object.values()`.
- Si ves `[]` piensa en **array** → usa mucho `for`, `forEach()` y `for...of`.